Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
ДИСЦИПЛИНЫ
«ОСНОВЫ НЕЙРОННЫХ СЕТЕЙ»
ВАРИАНТ № 2

Выполнил:
Левашев Тимур Рашидович
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Проверил:
Доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии Воронкин Р.А

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема работы: «Исследование архитектур рекуррентных сетей в задачах обработки естественного языка (NLP)»

Цель работы: Изучить принципы построения и обучения рекуррентных (RNN) нейронных сетей, провести сравнительный анализ их эффективности при классификации текстовых данных и исследовать влияние гиперпараметров (размер словаря, длина окна) на качество обучения модели.

Порядок выполнения работы

Ссылка на Git репозиторий: https://github.com/mazy99/neural-networks_5

1 Исследование влияния параметров словаря на точность классификации (Lite)

1.1 Подключение необходимых для работы библиотек

Листинг 1.1 – Подключение библиотек

```
# Работа с массивами данных
import numpy as np


# Класс для конструирования последовательной модели нейронной сети
from tensorflow.keras.models import Sequential

# Основные слои
from tensorflow.keras.layers import Dense, Dropout, SpatialDropout1D,
BatchNormalization, Embedding, Flatten, Activation
from tensorflow.keras.layers import SimpleRNN, GRU, LSTM, Bidirectional, Conv1D,
MaxPooling1D, GlobalMaxPooling1D, Input

# Токенизатор для преобразование текстов в последовательности
from tensorflow.keras.preprocessing.text import Tokenizer

# Рисование схемы модели
from tensorflow.keras.utils import plot_model

# Матрица ошибок классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
# Загрузка датасетов из облака google
import gdown

# Функции операционной системы
import os

# Регулярные выражения
import re

# Отрисовка графиков
import matplotlib.pyplot as plt

# Вывод объектов в ячейке colab
from IPython.display import display

from tensorflow.keras.callbacks import EarlyStopping

%matplotlib inline
```

1.2 Скачивание и распаковка датасета

Листинг 1.2 – Скачивание и распаковка датасета

```
# Загрузим датасет из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l7/writ
ers.zip', None, quiet=True)
# Распакуем архив в папку writers
!unzip -o writers.zip -d writers/
```

```
Archive:  writers.zip
  inflating: writers/(Клиффорд_Саймак) Обучающая_5 вместе.txt
  inflating: writers/(Клиффорд_Саймак) Тестовая_2 вместе.txt
  inflating: writers/(Макс Фрай) Обучающая_5 вместе.txt
  inflating: writers/(Макс Фрай) Тестовая_2 вместе.txt
  inflating: writers/(О. Генри) Обучающая_50 вместе.txt
  inflating: writers/(О. Генри) Тестовая_20 вместе.txt
  inflating: writers/(Рэй Брэдберри) Обучающая_22 вместе.txt
  inflating: writers/(Рэй Брэдберри) Тестовая_8 вместе.txt
  inflating: writers/(Стругацкие) Обучающая_5 вместе.txt
  inflating: writers/(Стругацкие) Тестовая_2 вместе.txt
  inflating: writers/(Булгаков) Обучающая_5 вместе.txt
  inflating: writers/(Булгаков) Тестовая_2 вместе.txt
```

Рисунок 1.1 – Результат распаковки архива

1.3 Разбиение датасета на обучающую и тестовую выборки и загрузка их в массивы

Листинг 1.3 – Разбиение и загрузка датасета в массивы

```
# Подготовим пустые списки

CLASS_LIST = []
text_train = []
text_test = []

file_list = sorted(os.listdir(FILE_DIR))

for file_name in file_list:
    m = re.match(r'^\((.+?)\)\s+(.+?)(?:_|$)', file_name)
    if not m:
        continue

    class_name = m.group(1)
    subset_name = m.group(2).lower()

    is_train = SIG_TRAIN in subset_name
    is_test = SIG_TEST in subset_name

    if not (is_train or is_test):
        continue

    if class_name not in CLASS_LIST:
        print(f'Добавление класса "{class_name}"')
        CLASS_LIST.append(class_name)
        text_train.append('')
        text_test.append('')

    cls = CLASS_LIST.index(class_name)
    print(f'Добавление файла "{file_name}" в класс "{CLASS_LIST[cls]}",
{subset_name} выборка.')

    file_path = os.path.join(FILE_DIR, file_name)
    with open(file_path, 'r', encoding='utf-8-sig') as f:
        text = f.read()

    subset = text_train if is_train else text_test
    subset[cls] += ' ' + text.replace('\n', ' ').strip()
```

```
Добавление класса "Булгаков"
Добавление файла "(Булгаков) Обучающая_5 вместе.txt" в класс "Булгаков", обучающая выборка.
Добавление файла "(Булгаков) Тестовая_2 вместе.txt" в класс "Булгаков", тестовая выборка.
Добавление класса "Клиффорд_Саймак"
Добавление файла "(Клиффорд_Саймак) Обучающая_5 вместе.txt" в класс "Клиффорд_Саймак", обучающая выборка.
Добавление файла "(Клиффорд_Саймак) Тестовая_2 вместе.txt" в класс "Клиффорд_Саймак", тестовая выборка.
Добавление класса "Макс Фрай"
Добавление файла "(Макс Фрай) Обучающая_5 вместе.txt" в класс "Макс Фрай", обучающая выборка.
Добавление файла "(Макс Фрай) Тестовая_2 вместе.txt" в класс "Макс Фрай", тестовая выборка.
Добавление класса "О. Генри"
Добавление файла "(О. Генри) Обучающая_50 вместе.txt" в класс "О. Генри", обучающая выборка.
Добавление файла "(О. Генри) Тестовая_20 вместе.txt" в класс "О. Генри", тестовая выборка.
Добавление класса "Рэй Брэдберри"
Добавление файла "(Рэй Брэдберри) Обучающая_22 вместе.txt" в класс "Рэй Брэдберри", обучающая выборка.
Добавление файла "(Рэй Брэдберри) Тестовая_8 вместе.txt" в класс "Рэй Брэдберри", тестовая выборка.
Добавление класса "Стругацкие"
Добавление файла "(Стругацкие) Обучающая_5 вместе.txt" в класс "Стругацкие", обучающая выборка.
Добавление файла "(Стругацкие) Тестовая_2 вместе.txt" в класс "Стругацкие", тестовая выборка.
```

Рисунок 1.2 – Результат загрузки

1.4 Преобразование текстов в последовательности

Листинг 1.4 – Настройка параметры словаря и преобразование текста в последовательности

```
# Задание параметров преобразования
VOCAB_SIZE = 20000                        # Объем словаря для токенизатора
WIN_SIZE   = 500                        # Длина отрезка текста (окна) в словах
WIN_HOP    = 20                         # Шаг окна разбиения текста на векторы
EPOCHS     = 50                           # Количество эпох обучения модели
BATCH_SIZE = 128                          # Размер батча для обучения модели
t = Tokenizer(num_words=VOCAB_SIZE)           # Создадим токенизатор для
преобразования текстов в последовательности
t.fit_on_texts(text_train)                   # Построим словарь токенизатора на
обучающей выборке
items = list(t.word_index.items())
print(f'Всего слов в словаре: {len(items)}')  # Выведем количество слов в словаре
токенизатора
print(f'Первые 20 слов в словаре: {items[:20]}')  # Выведем первые 20 слов в
словаре токенизатора
```

```
Всего слов в словаре: 146934
Первые 20 слов в словаре: [('и', 1), ('-', 2), ('в', 3), ('не', 4), ('я', 5), ('что', 6), ('на', 7), ('—', 8), ('с', 9), ('он', 10), ('а', 1
```

Рисунок 1.3 – Результат тонизации последовательностей

1.5 Вычисление статистик по текстам

Листинг 1.5 – Статистика по текстам

```
# Функция вывода статистики по текстам
def print_text_stats(title, texts, sequences, class_labels=CLASS_LIST):
    # Суммарное количество символов и слов в тексте
    chars = 0
    words = 0

    print(f'Статистика по {title} текстам:')

    # Вывод итогов по всем классам данного набора текстов и их
последовательностей индексов
    for cls in range(len(class_labels)):
        print('{:<15} {:9} символов,{:8} слов'.format(class_labels[cls],
                                                      len(texts[cls]),
                                                      len(sequences[cls])))
        chars += len(texts[cls])
        words += len(sequences[cls])

    print('----')
    print('{:<15} {:9} символов,{:8} слов\n'.format('В сумме', chars, words))

# Вывод итогов по текстам
print_text_stats('обучающим', text_train, seq_train)
print_text_stats('тестовым', text_test, seq_test)
```

```
Статистика по обучающим текстам:
Булгаков          1765647 символов,  222966 слов
Клиффорд_Саймак   1609507 символов,  228982 слов
Макс Фрай         3700010 символов,  529816 слов
О. Генри          1049518 символов,  133912 слов
Рэй Брэдберри     1386454 символов,  187195 слов
Стругацкие        2042465 символов,  280673 слов
----
В сумме          11553601 символов, 1583544 слов

Статистика по тестовым текстам:
Булгаков           875042 символов,  111363 слов
Клиффорд_Саймак    318811 символов,   44157 слов
Макс Фрай         1278191 символов,  173054 слов
О. Генри           349662 символов,   42821 слов
Рэй Брэдберри      868673 символов,  109553 слов
Стругацкие         704846 символов,   91852 слов
----
В сумме           4395225 символов,  572800 слов
```

Рисунок 1.4 – Статистика по текстам

1.6 Формирование обучающей и тестовой выборки в виде массивов отрезков текста и классов

Листинг 1.6 – Формирование обучающей и тестовой выборки

```
# Функция разбиения последовательности на отрезки скользящим окном
# На входе - последовательность индексов, размер окна, шаг окна
def split_sequence(sequence, win_size, hop):
    # Последовательность разбивается на части до последнего полного окна
    return [sequence[i:i + win_size] for i in range(0, len(sequence) - win_size +
1, hop)]


# Функция формирования выборок из последовательностей индексов
# формирует выборку отрезков и соответствующих им меток классов в виде one hot
encoding
def vectorize_sequence(seq_list, win_size, hop):
    # В списке последовательности следуют в порядке их классов
    # Всего последовательностей в списке ровно столько, сколько классов
    class_count = len(seq_list)

    # Списки для исходных векторов и категориальных меток класса
    x, y = [], []

    # Для каждого класса:
    for cls in range(class_count):
        # Разбиение последовательности класса cls на отрезки
        vectors = split_sequence(seq_list[cls], win_size, hop)
        # Добавление отрезков в выборку
        x += vectors
        # Для всех отрезков класса cls добавление меток класса в виде OHE
        y += [cls] * len(vectors)

    # Возврат результатов как numpy-массивов
    return np.array(x), np.array(y)
x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
    # Формирование тестовой выборки
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

    # Проверка формы сформированных данных
print(x_train.shape, y_train.shape)
print(x_test.shape, y_test.shape)
print(x_train[0])
```

```
(79030, 500) (79030,)
(28493, 500) (28493,)
[ 3730  8278  1525 15522  1572   358  8694  1271     1   125 14311   702
 11513 10176 17052     3   278  1058  4699   634     9  6976    16  2820
     8    54     8  1173     8  1690  1620     1    97  3427   940    18
     3  5547 18908     9  3891   531  6203    44   491     1  8695   491
    18  3025    33   513    27  4700   560  2397    44    10  2865  3497
    11  4701     1   594  1947     3  1222   794   392  1876  5161     8
     1  2434 18909  4331  3026    15   853     1     3     1     3 14312
   945 15523    12  8696     1  3428  3731     4  4098    12     3 10837
   973  7246    41  1355   327  9111  4701     1 17053  1261 12279  4449
    72    27    26  3138   106   491   131    99    12  2287  1050  3578
     9 15524  3805     1     3   569  1210    42  1312  1075  4450  3806
   756  3807   131  3732     1  7912   451     7 11514     3   276  4099
     3 11515   973  6977     9  2866  1541    18 18910     7  8279     3
  2507  4564  6204  2463  3027     6     7    42   769    44 13182  1174
     1  8280   532   340  5762  3733    33 10177     1 17054 15525     1
    17  7573     1   570     1 18911   248  3207    50   930 18912  6445
     7  1645   249  6701   245  1122  4450  1050  3808     1  4332     3
   459     1   590  3498  4333     9     7  3809  8281   794    17   826
  1356 18913 15526  3208   118    18 13183 18914  3499  1252  1063  2596
    10    47     7     7     3 10838 17055    72 10839     1  6446   692
   636    24     6   397 15527   269  3028 10178   769    42    16    42
  4565  6702     3  3500   634   636  2005     4  3977    11    95   590
    37     4   250     6    16     6    48   582   145    21    12   204
...
  1663  7576     9     7  2135     6 15531  5164     3  5165    69  2694
     7   387     9  4833 12280     9  8284 13187 10180  7247     8    16
  1342 18918     1  5166  4704  8699    16    14   769     3   300 15532
    75  6206  3979     1    43  4226     1 11517]
```

Рисунок 1.5 – Результат формирования выборок

1.7 Создание и обучение нейронной сети

Листинг 1.7 – Создание и обучение нейронной сети

```
model_LSTM = Sequential([
    Input(shape=(WIN_SIZE,)),
    Embedding(VOCAB_SIZE, 50),
    SpatialDropout1D(0.4),
    BatchNormalization(),

    Bidirectional(LSTM(8, return_sequences=True)),
    Bidirectional(LSTM(8, return_sequences=True)),
    Dropout(0.3),
    BatchNormalization(),

    GRU(16, return_sequences=True, reset_after=True),
    GRU(16, reset_after=True),
    Dropout(0.3),
    BatchNormalization(),

    Dense(200, activation='relu'),
    Dropout(0.3),
```

```
    BatchNormalization(),
    Dense(CLASS_COUNT, activation='softmax')
]
)




compile_train_eval_model(model_LSTM,
                         x_train, y_train,
                         x_test, y_test,
                         optimizer='rmsprop',
                         epochs=EPOCHS,
                         batch_size=BATCH_SIZE,
                         class_labels=CLASS_LIST,
                         title='Embedding(50) + BLSTM(8)x2 + GRU(16)x2 +
Dense(200)')
```

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 500, 50)	1,000,000
spatial_dropout1d_3 (SpatialDropout1D)	(None, 500, 50)	0
batch_normalization_12 (BatchNormalization)	(None, 500, 50)	200
bidirectional_6 (Bidirectional)	(None, 500, 16)	3,776
bidirectional_7 (Bidirectional)	(None, 500, 16)	1,600
dropout_9 (Dropout)	(None, 500, 16)	0
batch_normalization_13 (BatchNormalization)	(None, 500, 16)	64
gru_6 (GRU)	(None, 500, 16)	1,632
gru_7 (GRU)	(None, 16)	1,632
dropout_10 (Dropout)	(None, 16)	0
batch_normalization_14 (BatchNormalization)	(None, 16)	64
dense_6 (Dense)	(None, 200)	3,400
dropout_11 (Dropout)	(None, 200)	0
batch_normalization_15 (BatchNormalization)	(None, 200)	800
dense_7 (Dense)	(None, 6)	1,206

Рисунок 1.6 – Структура модели

```
Epoch 1/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 13s 129ms/step - accuracy: 0.2441 - loss: 2.0198 - val_accuracy: 0.3026 - val_loss: 1.7276
Epoch 2/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 107ms/step - accuracy: 0.3674 - loss: 1.6433 - val_accuracy: 0.3026 - val_loss: 1.6657
Epoch 3/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 118ms/step - accuracy: 0.5070 - loss: 1.2713 - val_accuracy: 0.2293 - val_loss: 1.7462
Epoch 4/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 113ms/step - accuracy: 0.6148 - loss: 1.0055 - val_accuracy: 0.4348 - val_loss: 1.5897
Epoch 5/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 112ms/step - accuracy: 0.7019 - loss: 0.8022 - val_accuracy: 0.4691 - val_loss: 1.4480
Epoch 6/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 117ms/step - accuracy: 0.7822 - loss: 0.6022 - val_accuracy: 0.4898 - val_loss: 2.3956
Epoch 7/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 107ms/step - accuracy: 0.8477 - loss: 0.4309 - val_accuracy: 0.5266 - val_loss: 2.5696
Epoch 8/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 119ms/step - accuracy: 0.9025 - loss: 0.3000 - val_accuracy: 0.5505 - val_loss: 2.2745
Epoch 9/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 107ms/step - accuracy: 0.9279 - loss: 0.2271 - val_accuracy: 0.6392 - val_loss: 1.6920
Epoch 10/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 117ms/step - accuracy: 0.9505 - loss: 0.1560 - val_accuracy: 0.6869 - val_loss: 1.3787
Epoch 11/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 110ms/step - accuracy: 0.9617 - loss: 0.1317 - val_accuracy: 0.5999 - val_loss: 2.4459
Epoch 12/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 111ms/step - accuracy: 0.9736 - loss: 0.0892 - val_accuracy: 0.6795 - val_loss: 1.6070
Epoch 13/50
...
Epoch 14/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 108ms/step - accuracy: 0.9752 - loss: 0.0866 - val_accuracy: 0.6974 - val_loss: 1.5556
Epoch 15/50
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 119ms/step - accuracy: 0.9809 - loss: 0.0629 - val_accuracy: 0.6971 - val_loss: 1.6048
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Рисунок 1.7 – Обучение модели

Рисунок 1.8 – Матрица ошибок

```
--------------------------------------------------------------------------------
Accuracy: 0.6869
Булгаков              → Булгаков              | OK
Клиффорд_Саймак       → О. Генри              | ERROR
Макс Фрай             → Макс Фрай             | OK
О. Генри              → О. Генри              | OK
Рэй Брэдберри         → Рэй Брэдберри         | OK
Стругацкие            → Стругацкие            | OK

Mean class accuracy: 0.6596
```

Рисунок 1.9 – Средняя точность

1.8 Изменение размера словаря tokenaizera (VOCAB_SIZE) на 40000, при этом оставил WIN_SIZE=1000, WIN_HOP=100

```
Epoch 1/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 39s 213ms/step - accuracy: 0.2643 - loss: 1.9432 - val_accuracy: 0.3015 - val_loss: 1.7294
Epoch 2/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 27s 202ms/step - accuracy: 0.4982 - loss: 1.3086 - val_accuracy: 0.1635 - val_loss: 1.9606
Epoch 3/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 27s 205ms/step - accuracy: 0.6877 - loss: 0.8409 - val_accuracy: 0.1904 - val_loss: 2.6794
Epoch 4/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 27s 205ms/step - accuracy: 0.7459 - loss: 0.6997 - val_accuracy: 0.5040 - val_loss: 1.8870
Epoch 5/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 27s 207ms/step - accuracy: 0.8826 - loss: 0.3499 - val_accuracy: 0.5983 - val_loss: 1.4539
Epoch 6/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 27s 205ms/step - accuracy: 0.9242 - loss: 0.2370 - val_accuracy: 0.5578 - val_loss: 1.8823
Epoch 7/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 27s 204ms/step - accuracy: 0.9436 - loss: 0.1820 - val_accuracy: 0.6736 - val_loss: 0.9829
Epoch 8/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 27s 205ms/step - accuracy: 0.9740 - loss: 0.0937 - val_accuracy: 0.6867 - val_loss: 1.2737
Epoch 9/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 28s 210ms/step - accuracy: 0.9907 - loss: 0.0395 - val_accuracy: 0.7261 - val_loss: 1.1818
Epoch 10/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 27s 205ms/step - accuracy: 0.9925 - loss: 0.0295 - val_accuracy: 0.7407 - val_loss: 1.2912
Epoch 11/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 27s 206ms/step - accuracy: 0.9933 - loss: 0.0272 - val_accuracy: 0.7387 - val_loss: 1.3208
Epoch 12/50
132/132 ━━━━━━━━━━━━━━━━━━━━ 27s 204ms/step - accuracy: 0.9977 - loss: 0.0135 - val_accuracy: 0.7453 - val_loss: 1.3299
```

Рисунок 1.10 – Результат обучение модели

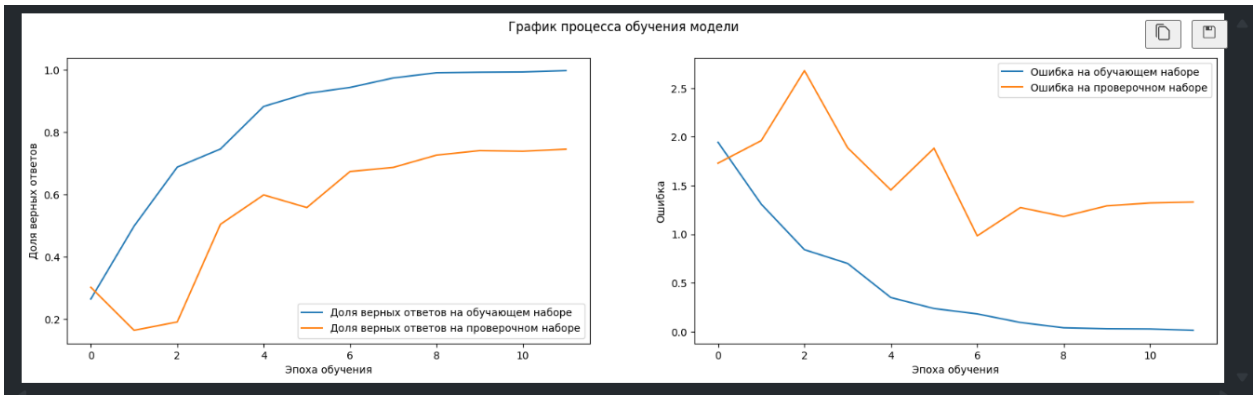

Рисунок 1.11 – График обучение модели

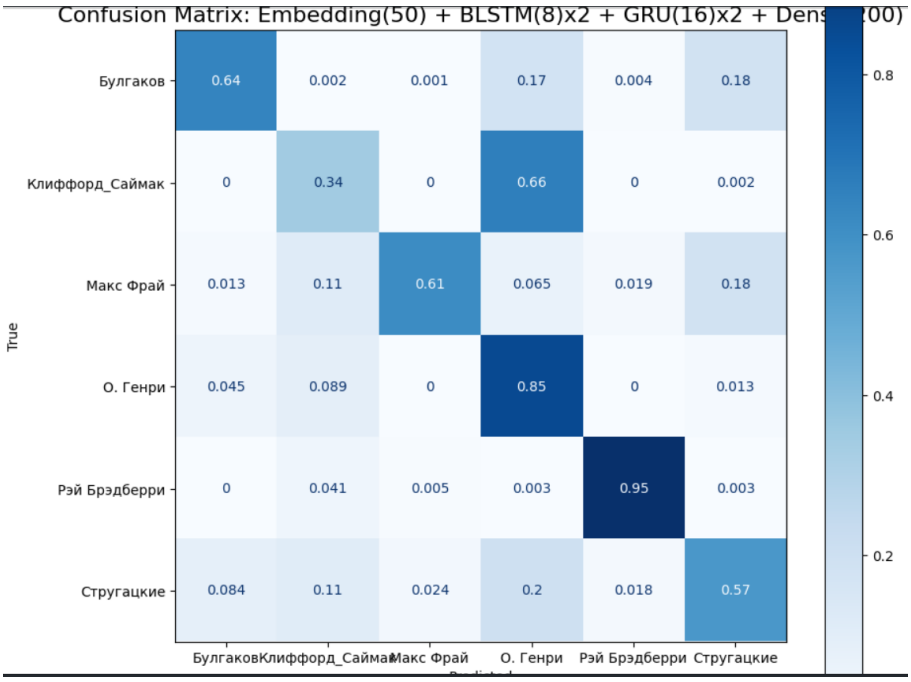

Рисунок 1.12 – Матрица ошибок модели

```
--------------------------------------------------------------------------
Accuracy: 0.6736
Булгаков              → Булгаков              | OK
Клиффорд_Саймак       → О. Генри              | ERROR
Макс Фрай             → Макс Фрай             | OK
О. Генри              → О. Генри              | OK
Рэй Брэдберри         → Рэй Брэдберри         | OK
Стругацкие            → Стругацкие            | OK

Mean class accuracy: 0.6606
```

Рисунок 1.13 – Средняя точность модели

1.9 Изменение длины отрезка текста 500 и шаг окна разбиения текста на 20, при этом оставил VOCAB_SIZE=20000

```
Epoch 1/50
618/618 ━━━━━━━━━━━━━━━━━━━━ 78s 116ms/step - accuracy: 0.5339 - loss: 1.2650 - val_accuracy: 0.5070 - val_loss: 1.3951
Epoch 2/50
618/618 ━━━━━━━━━━━━━━━━━━━━ 72s 116ms/step - accuracy: 0.8589 - loss: 0.3720 - val_accuracy: 0.6223 - val_loss: 1.3824
Epoch 3/50
618/618 ━━━━━━━━━━━━━━━━━━━━ 71s 115ms/step - accuracy: 0.9786 - loss: 0.0697 - val_accuracy: 0.6525 - val_loss: 1.6936
Epoch 4/50
618/618 ━━━━━━━━━━━━━━━━━━━━ 69s 112ms/step - accuracy: 0.9888 - loss: 0.0387 - val_accuracy: 0.5946 - val_loss: 2.1744
Epoch 5/50
618/618 ━━━━━━━━━━━━━━━━━━━━ 70s 113ms/step - accuracy: 0.9927 - loss: 0.0249 - val_accuracy: 0.6375 - val_loss: 2.1388
Epoch 6/50
618/618 ━━━━━━━━━━━━━━━━━━━━ 71s 115ms/step - accuracy: 0.9941 - loss: 0.0211 - val_accuracy: 0.6234 - val_loss: 2.5117
Epoch 7/50
618/618 ━━━━━━━━━━━━━━━━━━━━ 73s 118ms/step - accuracy: 0.9957 - loss: 0.0162 - val_accuracy: 0.6548 - val_loss: 2.3641
```

Рисунок 1.14 – Результат обучение модели

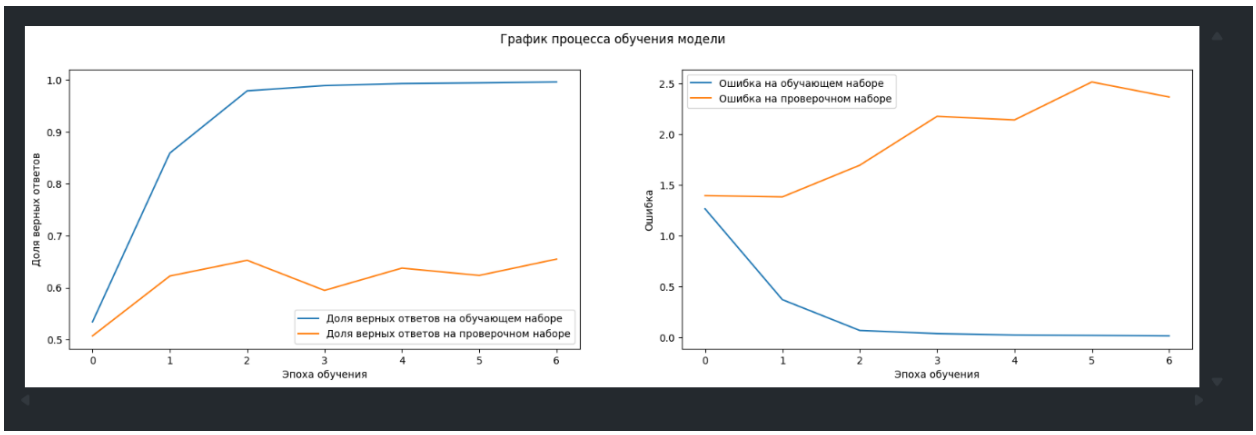

Рисунок 1.15 – График обучение модели

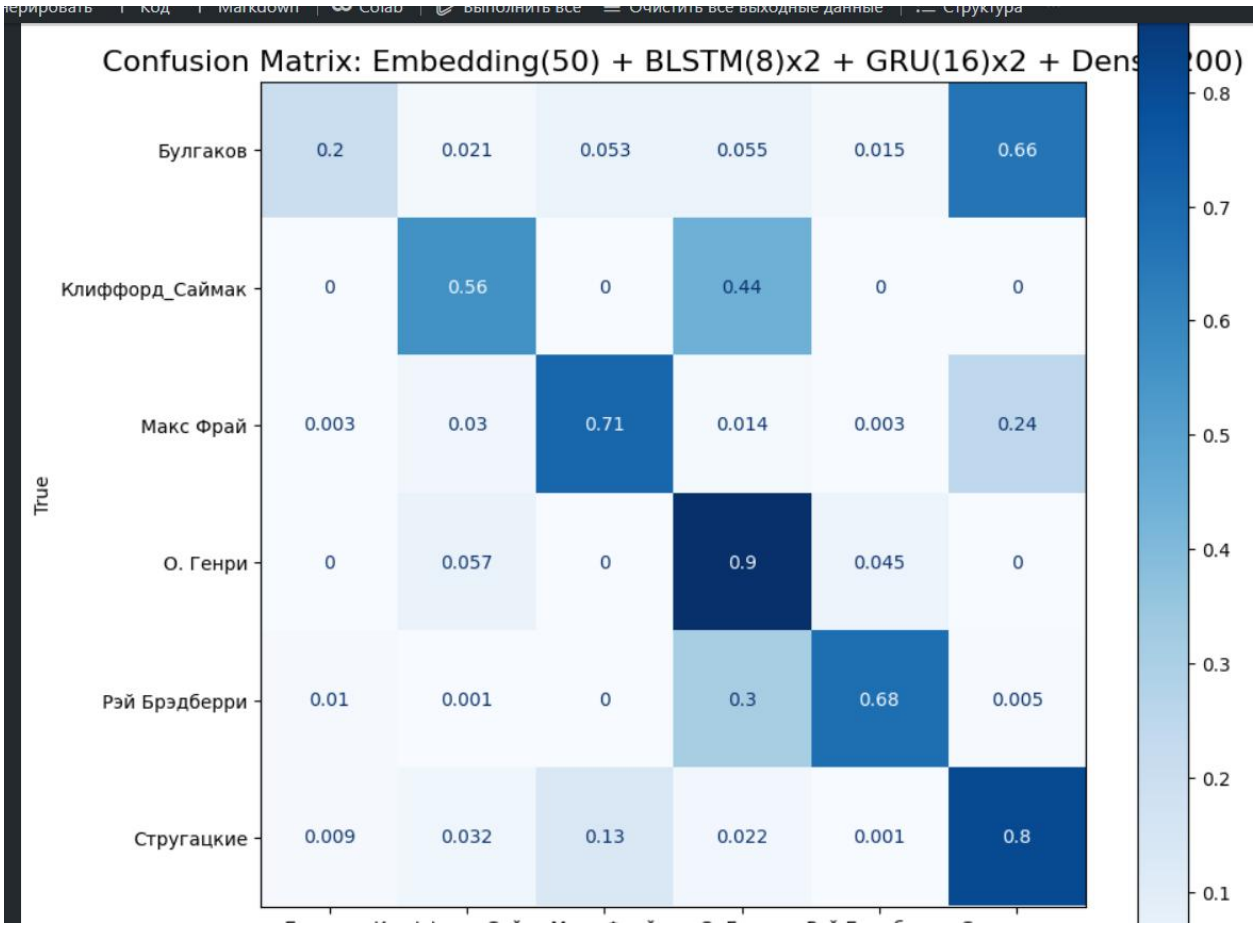

Рисунок 1.16 – Матрица ошибок модели

```
--------------------------------------------------------------------------------
Accuracy: 0.6223
Булгаков            → Стругацкие          | ERROR
Клиффорд_Саймак     → Клиффорд_Саймак     | OK
Макс Фрай           → Макс Фрай           | OK
О. Генри            → О. Генри            | OK
Рэй Брэдберри       → Рэй Брэдберри       | OK
Стругацкие          → Стругацкие          | OK

Mean class accuracy: 0.6415
```

Рисунок 1.17 – Средняя точность модели

2 Разработка нейросетевого классификатора медицинских болезней (Pro)

2.1 Подключение необходимых библиотек для выполнения задания

Листинг 2.1 – Импорт необходимых библиотек

```
# Работа с массивами данных
import numpy as np



# Класс для конструирования последовательной модели нейронной сети
from tensorflow.keras.models import Sequential

# Основные слои
from tensorflow.keras.layers import Dense, Dropout, SpatialDropout1D,
BatchNormalization, Embedding, Flatten, Activation
from tensorflow.keras.layers import SimpleRNN, GRU, LSTM, Bidirectional, Conv1D,
MaxPooling1D, GlobalMaxPooling1D,  Input

# Токенизатор для преобразование текстов в последовательности
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.model_selection import train_test_split

# Рисование схемы модели
from tensorflow.keras.utils import plot_model

# Матрица ошибок классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

```
from sklearn.utils.class_weight import compute_class_weight


# Функции операционной системы
import os



# Отрисовка графиков
import matplotlib.pyplot as plt

# Вывод объектов в ячейке colab
from IPython.display import display

%matplotlib inline
```

2.2 Скачивание и распаковка архива с датасетом

Листинг 2.2 – Скачивание и распаковка архива

```
# Скачаем архив с симптомами болезней
import gdown
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l8/dise
ases.zip', None, quiet=True)
# Распакум архив
!unzip -o diseases.zip
```

```
Archive:  diseases.zip
   creating: dis/
  inflating: dis/Аппендицит.txt
  inflating: dis/Гастрит.txt
  inflating: dis/Гепатит.txt
  inflating: dis/Дуоденит.txt
  inflating: dis/Колит.txt
  inflating: dis/Панкреатит.txt
  inflating: dis/Холицестит.txt
  inflating: dis/Эзофагит.txt
  inflating: dis/Энтерит.txt
  inflating: dis/Язва.txt
```

Рисунок 2.1 – Результат распаковки архива

2.3 Загрузка содержимого файлов в массивы

Листинг 2.3 – Загрузка текстов в массивы

```
# Подготовим пустые списки

CLASS_LIST = []  # Список классов
text_train = []  # Список для оучающей выборки
text_test = []   # Список для тестовой выборки

# Зададим коэффициент разделения текста на обучающую и текстовую выборки
split_coef = 0.8

# Получим списки файлов в папке
file_list = os.listdir(FILE_DIR)

for file_name in file_list:
    m = file_name.split('.') # Разделим имя файла и расширение
    class_name = m[0]        # Из имени файла получим название класса
    ext = m[1]               # Выделим расширение файла

    if ext=='txt':                                        # Если расширение txt
то берем файл в работу
        if class_name not in CLASS_LIST:                  # Проверим, есть уже
такой класс в списке
            print(f'Добавление класса "{class_name}"')     # Выведем имя нового
класса
            CLASS_LIST.append(class_name)                 # Добавим новый класс
в списоккласса "{class_name}"')

        cls =
CLASS_LIST.index(class_name)                                      # Получим
индекс (номер) нового класса
        print(f'Добавление файла "{file_name}" в класс
"{CLASS_LIST[cls]}"')      # Сообщим о появлении нового класса

        with open(f'{FILE_DIR}/{file_name}', 'r') as f: # Откроем файл на чтение
            text =
f.read()                                                     # Загрузка
содержимого файла в строку
            text = text.replace('\n', ' ').split('
')                            # Уберем символы перевода строк, получим список
слов
            text_len=len(text)
 # Найдем количество прочитанных слов
            text_train.append('
'.join(text[:int(text_len*split_coef)]))          # Выделим часть файла в
обучающую выборку
```

```
            text_test.append('
'.join(text[int(text_len*split_coef):]))          # Выделим часть файла в
тестовую выборку
```

2.4 Настройка параметров словаря, преобразование текстов в последовательности

Листинг 2.4 – Преобразование текстов в последовательности

```
# Задание параметров преобразования
VOCAB_SIZE = 10000                        # Объем словаря для токенизатора
WIN_SIZE   = 50                         # Длина отрезка текста (окна) в словах
WIN_HOP    = 2                          # Шаг окна разбиения текста на векторы
EPOCHS     = 30                           # Количество эпох обучения модели
BATCH_SIZE = 8                          # Размер батча для обучения модели
t = Tokenizer(num_words=VOCAB_SIZE)           # Создадим токенизатор для
преобразования текстов в последовательности
t.fit_on_texts(text_train)                   # Построим словарь токенизатора на
обучающей выборке
items = list(t.word_index.items())
print(f'Всего слов в словаре: {len(items)}')  # Выведем количество слов в словаре
токенизатора
print(f'Первые 20 слов в словаре: {items[:20]}')  # Выведем первые 20 слов в
словаре токенизатора
```

```
Всего слов в словаре: 1677
Первые 20 слов в словаре: [('в', 1), ('и', 2), ('боль', 3), ('или', 4), ('рвота', 5), ('тошнота', 6), ('боли', 7), ('живота', 8), ('животе',
```

Рисунок 2.2 – Результат токенизации текста

2.5 Преобразование выборок в последовательности

Листинг 2.5 – Преобразование выборок в последовательности

```
seq_train = t.texts_to_sequences(text_train)

seq_test = t.texts_to_sequences(text_test)

print("Фрагмент обучающего текста:")
print("В виде оригинального текста:              ", text_train[1][:101])
print("Он же в виде последовательности индексов: ", seq_train[1][:20])
print(len(seq_train))
print(len(seq_test))
```

```
Фрагмент обучающего текста:
В виде оригинального текста:              боль в эпигастральной области, тошнота, рвота,  общая слабость,  болезненность при пальпации в эг
Он же в виде последовательности индексов:  [3, 1, 46, 13, 6, 5, 41, 14, 82, 10, 184, 1, 46, 13, 14, 21, 22, 186, 2, 3]
10
10
```

Рисунок 2.3 – Преобразование выборок в последовательности

2.6 Формирование обучающей и тестовой выборок в виде массивов отрезков текстов и классов

Листинг 2.6 – Формирование обучающей и тестовой выборки

```
# Функция разбиения последовательности на отрезки скользящим окном
# На входе - последовательность индексов, размер окна, шаг окна
def split_sequence(sequence, win_size, hop):
    # Последовательность разбивается на части до последнего полного окна
    return [sequence[i:i + win_size] for i in range(0, len(sequence) - win_size +
1, hop)]


# Функция формирования выборок из последовательностей индексов
# формирует выборку отрезков и соответствующих им меток классов в виде one hot
encoding
def vectorize_sequence(seq_list, win_size, hop):
    # В списке последовательности следуют в порядке их классов
    # Всего последовательностей в списке ровно столько, сколько классов
    class_count = len(seq_list)

    # Списки для исходных векторов и категориальных меток класса
    x, y = [], []

    # Для каждого класса:
    for cls in range(class_count):
        # Разбиение последовательности класса cls на отрезки
        vectors = split_sequence(seq_list[cls], win_size, hop)
        # Добавление отрезков в выборку
        x += vectors
        # Для всех отрезков класса cls добавление меток класса в виде OHE
        y += [cls] * len(vectors)

    # Возврат результатов как numpy-массивов
    return np.array(x), np.array(y)
x_train, y_train = vectorize_sequence(seq_train, WIN_SIZE, WIN_HOP)
    # Формирование тестовой выборки
x_test, y_test = vectorize_sequence(seq_test, WIN_SIZE, WIN_HOP)

    # Проверка формы сформированных данных
print(x_train.shape, y_train.shape)
```

```
print(x_test.shape, y_test.shape)
print(x_train[0])
```

```
(3055, 50) (3055,)
(396, 50) (396,)
[741   7 330   1 453   8   5  54  21  22 108 255 123 331 332 124 256 173
 212 454   6   5  54  21  22  18   3   1 455  41  14 140 109 110 257 174
  31 125  20   8  33 456   1   9 175   3   1  38  32   8]
```

Рисунок 2.4 – Обучающая и тестовая выборка

2.7 Создание и обучение модели нейронной сети

Листинг 2.7 – Создание и обучение модели

```
model_LSTM = Sequential([
    Input(shape=(WIN_SIZE,)),

    # Увеличенная размерность вектора до 100 для лучшего разделения смыслов
    Embedding(input_dim=VOCAB_SIZE, output_dim=100),

    # SpatialDropout1D эффективнее для текстовых данных, так как выбрасывает
целые каналы
    SpatialDropout1D(0.3),
    BatchNormalization(),

    # Первый слой LSTM возвращает последовательность для второго слоя
    Bidirectional(LSTM(64, return_sequences=True)),
    Dropout(0.3),
    BatchNormalization(),

    # Второй слой LSTM агрегирует информацию
    Bidirectional(LSTM(32, return_sequences=False)),
    Dropout(0.3),

    BatchNormalization(),

    # Промежуточный мощный полносвязный слой
    Dense(64, activation='relu'),
    Dropout(0.4),
    BatchNormalization(),

    # Выходной слой на 10 классов
    Dense(CLASS_COUNT, activation='softmax')
])
```

```
compile_train_eval_model(model_LSTM,
                         x_train, y_train,
                         x_test, y_test,
                         epochs=EPOCHS,
                         optimizer='rmsprop',
                         batch_size=BATCH_SIZE,
                         class_labels=CLASS_LIST,
                         title='LSTM модель',)
```

Layer (type)	Output Shape	Param #
embedding_4 (Embedding)	(None, 50, 100)	1,000,000
spatial_dropout1d_4 (SpatialDropout1D)	(None, 50, 100)	0
batch_normalization_12 (BatchNormalization)	(None, 50, 100)	400
bidirectional_8 (Bidirectional)	(None, 50, 128)	84,480
dropout_12 (Dropout)	(None, 50, 128)	0
batch_normalization_13 (BatchNormalization)	(None, 50, 128)	512
bidirectional_9 (Bidirectional)	(None, 64)	41,216
dropout_13 (Dropout)	(None, 64)	0
batch_normalization_14 (BatchNormalization)	(None, 64)	256
dense_8 (Dense)	(None, 64)	4,160
dropout_14 (Dropout)	(None, 64)	0
batch_normalization_15 (BatchNormalization)	(None, 64)	256
dense_9 (Dense)	(None, 10)	650

Рисунок 2.5 – Структура модели

```
Epoch 1/30
382/382 ━━━━━━━━━━━━━━━━━━━━ 12s 21ms/step - accuracy: 0.2236 - loss: 2.2750 - val_accuracy: 0.4167 - val_loss: 1.8276
Epoch 2/30
382/382 ━━━━━━━━━━━━━━━━━━━━ 8s 22ms/step - accuracy: 0.5480 - loss: 1.2089 - val_accuracy: 0.5909 - val_loss: 1.3028
Epoch 3/30
382/382 ━━━━━━━━━━━━━━━━━━━━ 8s 22ms/step - accuracy: 0.8236 - loss: 0.5601 - val_accuracy: 0.6515 - val_loss: 1.0483
Epoch 4/30
382/382 ━━━━━━━━━━━━━━━━━━━━ 10s 20ms/step - accuracy: 0.8903 - loss: 0.3603 - val_accuracy: 0.6717 - val_loss: 1.5235
Epoch 5/30
382/382 ━━━━━━━━━━━━━━━━━━━━ 8s 20ms/step - accuracy: 0.9116 - loss: 0.2920 - val_accuracy: 0.6338 - val_loss: 2.0329
Epoch 6/30
382/382 ━━━━━━━━━━━━━━━━━━━━ 8s 21ms/step - accuracy: 0.9214 - loss: 0.2783 - val_accuracy: 0.7172 - val_loss: 1.3473
Epoch 7/30
382/382 ━━━━━━━━━━━━━━━━━━━━ 10s 21ms/step - accuracy: 0.9208 - loss: 0.2848 - val_accuracy: 0.7323 - val_loss: 1.2336
Epoch 8/30
382/382 ━━━━━━━━━━━━━━━━━━━━ 8s 20ms/step - accuracy: 0.9257 - loss: 0.2528 - val_accuracy: 0.7045 - val_loss: 1.5186
```

Рисунок 2.6 – Результат обучение модели

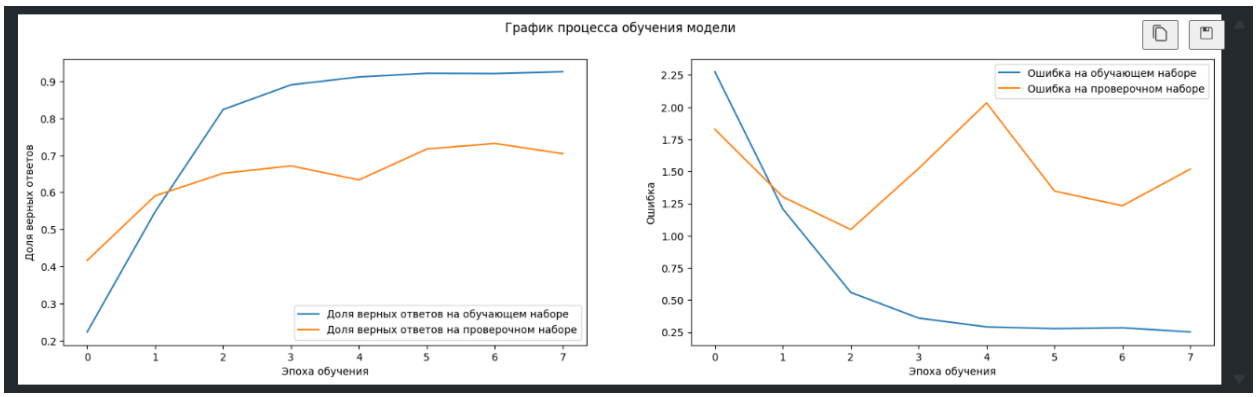

Рисунок 2.7 – График обучение модели

Рисунок 2.8 – Матрица ошибок модели

```
--------------------------------------------------------------------------------
Общая точность (Accuracy): 0.6515

Точность по классам (нормализованная):
  Энтерит          → 0.8140
  Дуоденит         → 0.4737
  Язва             → 1.0000
  Эзофагит         → 0.9688
  Колит            → 0.7222
  Гепатит          → 0.6552
  Аппендицит       → 1.0000
  Панкреатит       → 0.4337
  Холецистит       → 0.6829
  Гастрит          → 0.2750

Средняя точность по классам (macro): 0.7025
```

Рисунок 2.9 – Точность модели по каждому классу

3 Идентификация авторства и верификация собственного стиля (Ultra Pro)

3.1 Подключение необходимых для работы библиотек

Листинг 3.1 – Подключение библиотек

```
import os, re, numpy as np, pandas as pd
import gdown
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import classification_report, confusion_matrix,
ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import tensorflow as tf
import seaborn as sns
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SpatialDropout1D, Bidirectional,
LSTM, Dense, Dropout, BatchNormalization, SimpleRNN, GRU, GlobalMaxPooling1D
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.regularizers import l2
from tensorflow.keras.models import load_model

tf.random.set_seed(42)
```

```
np.random.seed(42)
```

3.2 Скачивание и распаковка архива с текстами

Листинг 3.2 – Скачивание и распаковка архива

```
url =
"https://storage.yandexcloud.net/aiueducation/Content/base/l7/20writers.zip"
gdown.download(url, "20writers.zip", quiet=False)
!mkdir -p my_ writers
!unzip -o 20writers.zip -d my_writers
```

```
Archive:  20writers.zip
  inflating: my_writers/Грибоедов.txt
  inflating: my_writers/Достоевский.txt
  inflating: my_writers/Каверин.txt
  inflating: my_writers/Катаев.txt
  inflating: my_writers/Куприн.txt
  inflating: my_writers/Лермонтов.txt
  inflating: my_writers/Лесков.txt
  inflating: my_writers/Носов.txt
  inflating: my_writers/Пастернак.txt
  inflating: my_writers/Пушкин.txt
  inflating: my_writers/Толстой.txt
  inflating: my_writers/Тургенев.txt
  inflating: my_writers/Чехов.txt
  inflating: my_writers/Шолохов.txt
  inflating: my_writers/Беляев.txt
  inflating: my_writers/Булгаков.txt
  inflating: my_writers/Васильев.txt
  inflating: my_writers/Гоголь.txt
  inflating: my_writers/Гончаров.txt
  inflating: my_writers/Горький.txt
```

Рисунок 3.1 – Результат распаковки архива

3.3 Чтение файлов и создание DataFrame (автор, текст)

Листинг 3.3 – Чтение файлов и создание Df

```
data = []
for filename in os.listdir("my_writers"):
    if filename.endswith(".txt"):
        author = os.path.splitext(filename)[0]
        with open(os.path.join("my_writers", filename), 'r', encoding='utf-8') as
f:
            text = f.read()
```

```
        data.append({"author": author, "text": text})
df = pd.DataFrame(data)

display(df.head())
```

	author	text
0	Горький	МАКАР ЧУДРА*\nС моря дул влажный, холодный ве...
1	Пушкин	\nВ одной из отдаленных наших губерний находи...
2	Каверин	Снегурочка и космополитизм\nРассказ\n\nЗадумы...
3	Васильев	А зори здесь тихие...1\n\nВ списках не значился...
4	Достоевский	СКАЗАНИЕ ОТЦА НАШЕГО АГАПИЯ,\n\nЗАЧЕМ ОСТАВЛЯ...

Рисунок 3.2 – Вывод первых 5 строк таблицы

3.4 Предобработка, очистка и лемматизация текста

Листинг 3.4 – Предобработка текста

```
def clean_only(text):
    # Оставляем буквы и основные знаки препинания, которые важны для стиля
    text = re.sub(r'[^а-яА-ЯёЁ\s\.!\?,:;\-]', ' ', text.lower())
    return ' '.join(text.split())

df['clean_text'] = df['text'].apply(clean_only)

display(df[['author', 'clean_text']].head())
```

	author	clean_text
0	Горький	макар чудра с моря дул влажный, холодный ветер...
1	Пушкин	в одной из отдаленных наших губерний находилос...
2	Каверин	снегурочка и космополитизм рассказ задумывалис...
3	Васильев	а зори здесь тихие в списках не значилсячасть ...
4	Достоевский	сказание отца нашего агапия, зачем оставляют с...

Рисунок 3.3 – Текста после очистки

3.5 Преобразование выборок в последовательности

Листинг 3.5 – Создание последовательностей

```
FRAG_LEN = 200
STEP = 100

def split_into_fragments(text, length=FRAG_LEN, step=STEP):
    words = text.split()
    frags = []
    for start in range(0, len(words)-length+1, step):
        frags.append(' '.join(words[start:start+length]))
    return frags

X, y = [], []
for _, row in df.iterrows():
    frags = split_into_fragments(row['clean_text'])
    X.extend(frags)
    y.extend([row['author']] * len(frags))

df_frag = pd.DataFrame({'text': X, 'author': y})

# Пересоздаем выборки
X_train, X_test, y_train, y_test = train_test_split(
    df_frag['text'], df_frag['author'],
    test_size=0.2, random_state=42, stratify=df_frag['author']
)

display(df_frag.head())
```

	text	author
0	макар чудра с моря дул влажный, холодный ветер...	Горький
1	позе, лицом ко мне, методически потягивал из с...	Горький
2	живут без тебя и проживут без тебя. разве ты д...	Горький
3	никто не знает. видишь, как человек пашет, и д...	Горький
4	а я, вот смотри, в пятьдесят восемь лет стольк...	Горький

Рисунок 3.4 – Пример последовательностей

3.6 Разделение последовательностей на обучающую и тестовую выборки

Листинг 3.6 – Разделение последовательностей

```
X_train, X_test, y_train, y_test = train_test_split(
    df_frag['text'], df_frag['author'],
    test_size=0.2, random_state=42, stratify=df_frag['author']
)

authors = sorted(df_frag['author'].unique())
author2id = {a:i for i,a in enumerate(authors)}
id2author = {i:a for a,i in author2id.items()}
num_classes = len(authors)

y_train_enc = [author2id[a] for a in y_train]
y_test_enc  = [author2id[a] for a in y_test]

print("Authors:", authors)
print("Number of classes:", num_classes)
print("Train samples:", len(X_train))
print("Test samples:", len(X_test))
```

```
Authors: ['Беляев', 'Булгаков', 'Васильев', 'Гоголь', 'Гончаров', 'Горький', 'Грибоедов', 'Достоевский', 'Каверин', 'Катаев', 'Куприн', 'Лер
Number of classes: 20
Train samples: 68692
Test samples: 17174
```

Рисунок 3.5 – Результат разделения

3.7 Токенизация и дополнение коротких последовательностей «пустыми» значениями

Листинг 3.7 – Токенизация последовательностей

```
VOCAB_SIZE = 10000  # ОПТИМИЗАЦИЯ: было 20000
MAX_LEN = 200       # ОПТИМИЗАЦИЯ: было 500 - ДОЛЖНО СООТВЕТСТВОВАТЬ FRAG_LEN!

tokenizer = Tokenizer(num_words=VOCAB_SIZE, oov_token="<OOV>", filters='')
tokenizer.fit_on_texts(X_train)

X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq  = tokenizer.texts_to_sequences(X_test)

X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LEN, padding='post',
truncating='post')
X_test_pad  = pad_sequences(X_test_seq,  maxlen=MAX_LEN, padding='post',
truncating='post')
```

3.8 Создание и обучение нейронной сети

Листинг 3.8 – Создание и обучение нейронной сети

```
def build_advanced_bilstm():
    model = Sequential([
        Embedding(VOCAB_SIZE, 128),
        SpatialDropout1D(0.2),

        # Один LSTM без BatchNormalization (быстрее!)
        Bidirectional(LSTM(32, return_sequences=False, dropout=0.2)),

        Dense(64, activation='relu'),
        Dropout(0.2),

        Dense(num_classes, activation='softmax')
    ])

    return model

class_weights = compute_class_weight('balanced', classes=np.unique(y_train_enc),
y=y_train_enc)
class_weight_dict = dict(enumerate(class_weights))

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True,
    min_delta=1e-4
)

history = model.fit(
    X_train_pad, np.array(y_train_enc),
    batch_size=32,  # КРИТИЧНО! было 256 - в 8 раз меньше = в 8 раз быстрее!
    epochs=20,
    validation_data=(X_test_pad, np.array(y_test_enc)),
    callbacks=[early_stop],
    class_weight=class_weight_dict,
    verbose=1
)
```

```
Epoch 1/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 58s 23ms/step - accuracy: 0.1886 - loss: 2.4407 - val_accuracy: 0.3685 - val_loss: 1.8026
Epoch 2/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 48s 22ms/step - accuracy: 0.4727 - loss: 1.4289 - val_accuracy: 0.6507 - val_loss: 1.0466
Epoch 3/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 49s 23ms/step - accuracy: 0.7225 - loss: 0.7481 - val_accuracy: 0.7964 - val_loss: 0.6669
Epoch 4/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 47s 22ms/step - accuracy: 0.8421 - loss: 0.4349 - val_accuracy: 0.8524 - val_loss: 0.5106
Epoch 5/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 47s 22ms/step - accuracy: 0.8951 - loss: 0.2908 - val_accuracy: 0.8720 - val_loss: 0.4553
Epoch 6/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 48s 22ms/step - accuracy: 0.9254 - loss: 0.2036 - val_accuracy: 0.8880 - val_loss: 0.4160
Epoch 7/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 48s 22ms/step - accuracy: 0.9413 - loss: 0.1585 - val_accuracy: 0.8990 - val_loss: 0.3815
Epoch 8/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 47s 22ms/step - accuracy: 0.9545 - loss: 0.1260 - val_accuracy: 0.8990 - val_loss: 0.4124
Epoch 9/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 83s 22ms/step - accuracy: 0.9613 - loss: 0.1056 - val_accuracy: 0.9050 - val_loss: 0.3904
Epoch 10/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 47s 22ms/step - accuracy: 0.9662 - loss: 0.0928 - val_accuracy: 0.9092 - val_loss: 0.3800
Epoch 11/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 45s 21ms/step - accuracy: 0.9734 - loss: 0.0728 - val_accuracy: 0.9113 - val_loss: 0.3786
Epoch 12/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 46s 22ms/step - accuracy: 0.9745 - loss: 0.0690 - val_accuracy: 0.9133 - val_loss: 0.3787
Epoch 13/20
...
Epoch 19/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 45s 21ms/step - accuracy: 0.9873 - loss: 0.0366 - val_accuracy: 0.9284 - val_loss: 0.3740
Epoch 20/20
2147/2147 ━━━━━━━━━━━━━━━━━━━━ 45s 21ms/step - accuracy: 0.9880 - loss: 0.0344 - val_accuracy: 0.9262 - val_loss: 0.3685
```

Рисунок 3.6 – Результат обучение модели

```
537/537 ━━━━━━━━━━━━━━━━━━━━ 4s 7ms/step
Accuracy: 0.9228
              precision    recall  f1-score   support

      Беляев       0.95      0.94      0.95       671
    Булгаков       0.95      0.92      0.93       600
    Васильев       0.89      0.92      0.90       883
      Гоголь       0.92      0.91      0.91       604
    Гончаров       0.95      0.93      0.94       974
     Горький       0.95      0.95      0.95       804
   Грибоедов       0.97      0.88      0.92       299
 Достоевский       0.92      0.90      0.91      1612
     Каверин       0.86      0.95      0.90       605
      Катаев       0.86      0.93      0.89       761
      Куприн       0.81      0.89      0.85       699
   Лермонтов       0.92      0.97      0.95       597
      Лесков       0.96      0.90      0.93       667
       Носов       0.94      0.97      0.95       756
   Пастернак       0.88      0.82      0.85       908
      Пушкин       0.92      0.91      0.91      1028
     Толстой       0.94      0.96      0.95      1037
    Тургенев       0.92      0.92      0.92       608
       Чехов       0.95      0.93      0.94      2068
     Шолохов       0.98      0.96      0.97       993

    accuracy                           0.92     17174
   macro avg       0.92      0.92      0.92     17174
weighted avg       0.92      0.92      0.92     17174
```

Рисунок 3.7 – Статистки precision, recall, f1-score

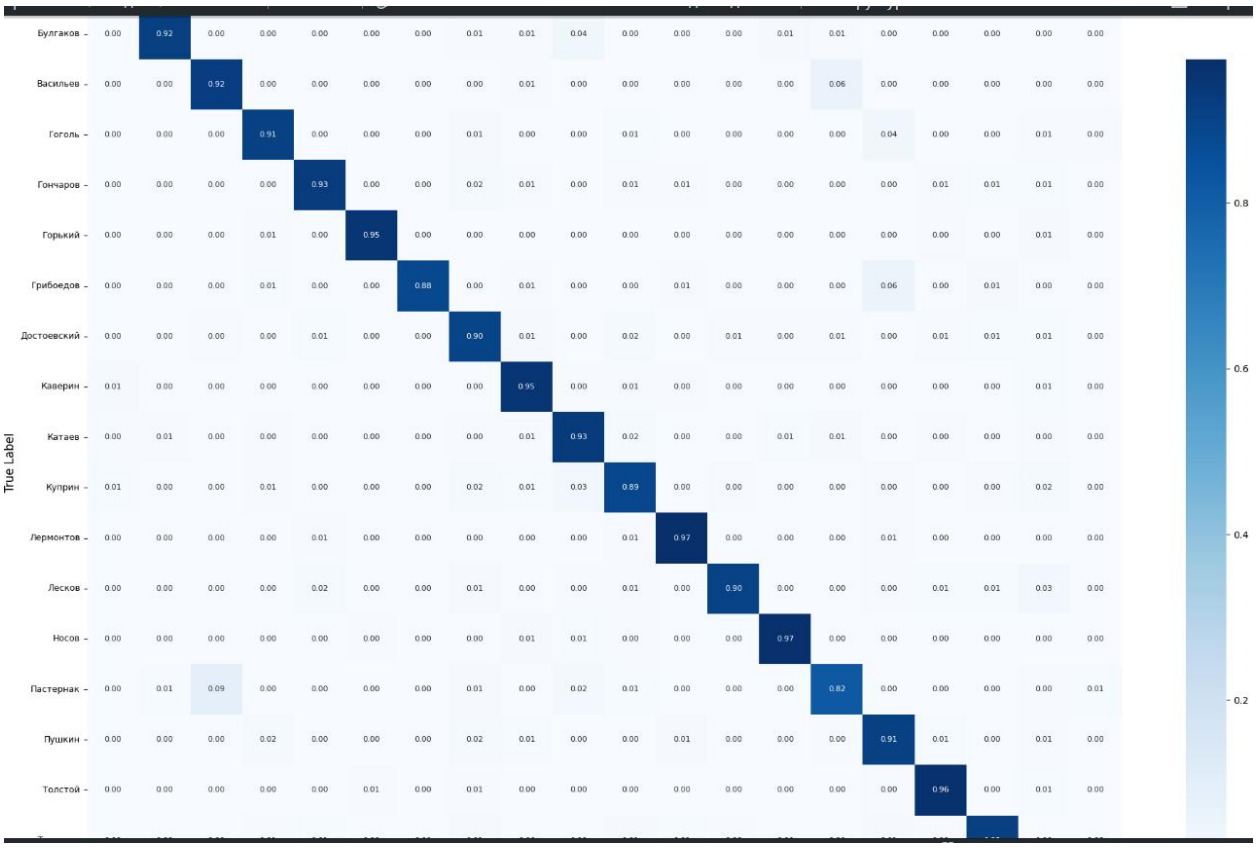

Рисунок 3.8 – Матрица ошибок модели

3.9 Оценка модели на собственном тексте

Листинг 3.9 – Оценка модели на собственном тексте

```
with open("my_text.txt", "r", encoding="utf-8") as f:
    text = f.read().lower()


FRAG_LEN = 200
STEP = 20


seq = tokenizer.texts_to_sequences([text])[0]

print("Длина последовательности:", len(seq))


samples = []

if len(seq) < FRAG_LEN:

    padded = pad_sequences(
        [seq],
```

```
        maxlen=FRAG_LEN,
        padding='post'
    )

    samples = padded

else:

    for i in range(0, len(seq) - FRAG_LEN, STEP):
        samples.append(seq[i:i + FRAG_LEN])

    samples = np.array(samples)

print("Количество окон:", len(samples))



pred = model.predict(samples)

mean_pred = pred.mean(axis=0)

author_index = np.argmax(mean_pred)
author_name = df['author'][author_index]


print("\nВероятности авторов:\n")

for i, author in enumerate(df['author']):
    print(f"{author}: {mean_pred[i] * 100:.2f}%")

print("\n======================")
print(f"Нейросеть считает, что автор текста — {author_name}")
print("======================")
```

```
Количество окон: 1
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step

Вероятности авторов:

Горький: 0.00%
Пушкин: 0.00%
Каверин: 0.00%
Васильев: 0.00%
Достоевский: 0.00%
Тургенев: 99.95%
Катаев: 0.00%
Булгаков: 0.00%
Лесков: 0.00%
Куприн: 0.00%
Гончаров: 0.00%
Шолохов: 0.00%
Грибоедов: 0.00%
Беляев: 0.03%
Носов: 0.00%
Гоголь: 0.00%
Пастернак: 0.00%
Чехов: 0.00%
Толстой: 0.01%
...

======================
Нейросеть считает, что автор текста – Тургенев
======================
```

Рисунок 3.9 – Оценка модели на собственном тексте

Текст, поданный в модель:

В тот вечер дождь шёл особенно долго. Сырой ветер стучал в оконные рамы, а старые часы в углу комнаты отсчитывали время так медленно, словно сами устали от однообразия дней. Иван Сергеевич сидел за письменным столом, не раскрывая книги, лежавшей перед ним уже третий час. Мысли его были заняты вовсе не чтением.

Ему казалось, что жизнь проходит мимо, тихо и незаметно, как проходят люди по улице в туманное утро: очертания видны, но стоит приглядеться – и всё исчезает. Когда-то он мечтал о службе, о путешествиях, о шумных беседах в столичных гостиных, однако теперь даже короткая поездка в соседний уезд представлялась ему делом утомительным и почти невозможным.

Из соседней комнаты доносился слабый звон посуды. Старуха-экономка, как и каждый вечер, готовила чай. Этот привычный звук внезапно вызвал в нём странное чувство спокойствия. Быть может, человек по-настоящему ценит не великие события, а именно такие тихие мгновения, которые раньше казались пустяком.

Он поднялся, подошёл к окну и долго смотрел на мокрую улицу. Одинокий фонарь освещал редких прохожих, спешивших скрыться от непогоды. Иван Сергеевич неожиданно улыбнулся. Впервые за многие месяцы ему подумалось, что завтрашний день ещё может принести что-то новое.

4 Выводы

В ходе выполнения работы были изучены принципы построения и обучения рекуррентных (RNN) нейронных сетей, проведён сравнительный анализ их эффективности при классификации текстовых данных и исследовано влияние гиперпараметров (размер словаря, длина окна) на качество обучения модели.